

基于 Huffman 编码的文档压缩系统

课程项目说明

适用对象：计算机科学与技术、软件工程及相关专业本科生

建议语言：C++（推荐），也可使用 Java / Python 等语言

项目性质：综合编程实践项目

总分：100 分（所有任务均为必做）

一、项目背景

1.1 什么是 Huffman 编码

Huffman 编码是一种经典的无损数据压缩算法，核心思想很简单：

出现频率高的字符用短编码，出现频率低的字符用长编码，这样整体占用的空间就变小了。

举个例子：假设一个文件里字母 **a** 出现了 1000 次，字母 **z** 只出现了 3 次。如果给 **a** 分配编码 **01**（2 位），给 **z** 分配编码 **110101**（6 位），相比每个字符都用 8 位（1 字节）存储，能节省大量空间。

1.2 Huffman 编码是怎么工作的

整个过程分四步：

1. **统计频率** — 扫描文件，统计每个字节（0x00-0xFF，共 256 种）出现了多少次。
2. **建树** — 把每个出现的字节作为一个“叶子节点”（权重=出现次数），反复取出权重最小的两个节点合并成一个新节点（权重=两者之和），直到只剩一个根节点。这就是 Huffman 树。
3. **生成编码** — 从根节点出发，走左边记 **0**，走右边记 **1**，到达某个叶子节点时，路径上的 01 串就是这个字节的 Huffman 编码。
4. **压缩 / 解压** — 编码时把每个字节替换成对应的 01 串写入文件；解压时从根节点出发，根据 01 串走到叶子节点还原字节。

直观理解：Huffman 树就像一张“翻译表”，把原始字节翻译成更短的 01 串，解压时再翻译回去。

1.3 你要做什么

从零实现一个命令行压缩工具：输入一个文件，输出压缩后的 **.huff** 文件；也能把 **.huff** 文件还原回原始文件。

二、你需要处理的文件

项目提供了 6 个测试文件，都在 **其他附件/** 目录下：

文件名	大小	内容	编码
1.txt	6 KB	纯英文文本	UTF-8
2.txt	18 KB	纯英文文本	UTF-8
3.txt	37 KB	纯英文文本	UTF-8
hamlet.txt	181 KB	莎士比亚《哈姆雷特》英文原文	UTF-8
斗破苍穹.txt	16 MB	中文小说（中英混合）	UTF-8
完美世界.txt	21 MB	中文小说（纯中文为主）	UTF-8

关于 UTF-8 编码

- 英文在 UTF-8 下和 ASCII 完全一样，每个字符占 **1 字节**。
- 中文在 UTF-8 下是变长的，一个常用汉字占 **3 字节**。
- 处理方式：按字节处理即可。** 把每个字节当成一个 0-255 的数值，用 `unsigned char` 读取，用一个 `count[256]` 数组统计每种字节出现了多少次。不管是英文还是中文，这个方法都能统一处理，不需要区分字符类型。

参考代码思路（来自 `中文处理（仅供参考）.md`）：

```
FILE* fp = fopen("test.txt", "rb");
unsigned char readbuff[1024];
int count[256] = {0};

while (true) {
    int read_size = fread(readbuff, 1, 1024, fp);
    if (read_size == 0) break;
    for (int i = 0; i < read_size; ++i) {
        count[readbuff[i]]++; // 用字节值做下标直接计数
    }
}
```

注意：本项目提供的文件仅用于课程学习，请不要传播或用于商业目的。

三、你的程序怎么运行（接口规范）

这是最重要的部分。 你的程序必须严格按以下格式接受命令，因为助教会用脚本进行批量自动测试。格式不对会被视为功能未完成。

你的程序编译后生成的可执行文件命名为 `Huffman`（Windows 下为 `Huffman.exe`）。

3.1 三种运行模式

模式一：压缩

```
Huffman -c <文件路径>
```

示例：

```
Huffman -c ./data/hamlet.txt
```

读取指定文件，统计频率、构建 Huffman 树、压缩，在同目录下生成以下 3 个文件：

生成的文件	命名规则	内容说明
压缩文件	原文件名.huff	二进制压缩文件
频率表	原文件名_freq.txt	文本文件，记录每种字节的出现频率
解压验证文件	原文件名_unzip.txt	压缩后立刻解压还原，用于验证正确性

即 `Huffman -c ./data/hamlet.txt` 会在 `./data/` 下生成：

- `hamlet.huff`
- `hamlet_freq.txt`
- `hamlet_unzip.txt`

模式二：解压

```
Huffman -x <文件路径>
```

示例：

```
Huffman -x ./data/hamlet.huff
```

读取 `.huff` 压缩文件，解析、重建 Huffman 树、解码，在同目录下生成：

生成的文件	命名规则	内容说明
解压文件	原文件名_unzip.txt	还原后的原始文本

即 `Huffman -x ./data/hamlet.huff` 会在 `./data/` 下生成 `hamlet_unzip.txt`。

模式三：自适应流式压缩

```
Huffman -a <目录路径>
```

示例：

```
Huffman -a ./data/adaptive_data/
```

模拟"数据流逐步到达"的场景。程序自动处理指定目录下的 1.txt、2.txt、3.txt（按文件名排序），每处理一个新文件，都会累积前面的频率来更新 Huffman 树。**一次运行完成全部三个阶段。**

阶段	处理的文件	Huffman 树依据	在同目录下生成的文件
一	1.txt	1.txt 自身的频率	1_adp.huff、1_freq.txt、1_unzip.txt
二	2.txt	1.txt + 2.txt 的累积频率	2_adp.huff、2_freq.txt、2_unzip.txt
三	3.txt	1.txt + 2.txt + 3.txt 的累积频率	3_adp.huff、3_freq.txt、3_unzip.txt

总共生成 9 个文件。

为什么这样做？ 真实场景（如网络传输）中数据是逐步到达的，编码表需要根据新数据动态更新。阶段二用 1+2 的累积频率建树来压缩 2.txt，是因为此时实际"见过"的数据是 1 和 2 的并集。

3.2 接口速查表

命令	作用	生成什么
Huffman -c <file>	压缩文件	<file>.huff、<file>_freq.txt、<file>_unzip.txt
Huffman -x <file>	解压文件	<file>_unzip.txt
Huffman -a <dir>	自适应流式压缩	9 个文件（见 3.1 模式三）

四、具体任务与评分（100 分）

4.1 基础压缩与解压（10 分）

你要做的：

- 实现 Huffman -c 和 Huffman -x 两个命令。
- 生成的文件命名和位置严格符合第三节的规范。
- 压缩再解压后的文件与原始文件**逐字节完全一致**。

_freq.txt 频率表格式：

```
字节值(十进制)  [字符]  出现次数
```

示例：

```
97  a  1240
101 e   890
32  .   560
10  .   430
...
```

- 按出现次数从高到低排序；次数相同按字节值从小到大排序。
 - 不可打印字符（控制字符、高位字节等）在字符列显示为 `.`。
- 扣分点：文件名不对、输出位置不对 → 此项不得分。

4.2 hamlet.txt 压缩 + 优先队列优化（25 分）

你要做的：

- 用 `Huffman -c` 压缩 `hamlet.txt`，确保解压后完全一致。
 - 构建 Huffman 树时，**必须使用优先队列（小根堆）**来找频率最小的两个节点，不能用每次线性扫描的方式。
- 为什么？ 如果每次都用 for 循环扫描整个数组找最小值（ $O(n^2)$ 复杂度），`hamlet.txt` 还勉强能跑，但后面的 16MB+ 中文文件就非常慢了。用堆可以把构建时间降到 $O(n \log n)$ 。

C++ 提示：用 `std::priority_queue` 配合 `std::greater` 改成小顶堆，或用 `std::multiset`。

扣分点：没用优先队列 / 堆优化 → 此项最多得 10 分。

4.3 自定义哈希校验文件一致性（20 分）

你要做的：

- 实现**两种不同的自定义散列函数**，用于计算文件的哈希值。
 - 用这两种哈希函数分别验证压缩前后文件是否一致（原始文件和 `_unzip.txt` 的哈希值应该完全相同）。
 - 在报告中完成分析。
- 重要：必须自己实现散列函数，不能用 `std::hash` 包装一下交差。

评分细分：

子项	分值	说明
两种自定义散列函数	8 分	每种 4 分，必须是自己写的算法
碰撞概率分析	4 分	分析两种哈希函数在本题文件大小下的碰撞概率
MD5(128位) vs 64位哈希	4 分	讨论为什么实际工程中 128 位哈希比 64 位更安全
哈希比较 vs 逐字节比较	4 分	分析两种校验方式的各自优缺点

4.4 中文大文件压缩（20 分）

你要做的：

- 用 `Huffman -c` 分别压缩 `斗破苍穹.txt`（16MB，中英混合）和 `完美世界.txt`（21MB，纯中文）。
- 确保解压后与原始文件完全一致。
- 保留 `.huff` 和 `_freq.txt` 文件。

4.5 自适应流式压缩（20 分）

你要做的：

- 实现 `Huffman -a <目录>` 命令。
- 程序一次运行，依次处理目录下的 `1.txt`、`2.txt`、`3.txt`。
- **关键是：**处理 `2.txt` 时基于 `1.txt + 2.txt` 的累积频率建树，处理 `3.txt` 基于 `1+2+3` 的累积频率建树。

直观理解：就像看视频——视频数据逐步到达，编码器需要根据“目前为止收到的所有数据”来动态调整压缩策略。

_freq.txt 说明：自适应模式下生成的 `_freq.txt` 体现的是**累积频率**（而非当前单个文件的频率），格式和普通模式完全一样。

4.6 信息熵与编码效率分析（5 分）

你要做的：

- 在报告中计算每个文件的 Shannon 信息熵和 Huffman 平均码长。
- 对比 `hamlet.txt`（纯英文）和 `斗破苍穹.txt`（中英混合）的编码效率，分析差异原因。

公式：

- **信息熵：** $H = -\sum (p_i \times \log_2(p_i))$

p_i = 字节 i 出现的概率。信息熵是理论上能达到的最短平均码长（压缩的理论极限）。

- **平均码长：** $L = \sum (p_i \times \text{len}(\text{code}_i))$

code_i = 字节 i 的 Huffman 编码。这是你的 Huffman 编码实际达到的平均码长。

- **编码效率：** $\eta = H / L \times 100\%$

越接近 100% 说明你的 Huffman 编码越接近理论最优。

报告中要分析：为什么中英混合文本的编码效率和纯英文不同？和字节分布有什么关系？

五、评分总表

编号	任务	分值	怎么算完成
1	基础压缩与解压	10	接口格式正确、文件命名正确、解压后与原文件一致
2	hamlet.txt + 优先队列	25	压缩解压一致、用了堆优化、频率表正确
3	自定义哈希校验	20	两种自实现哈希、碰撞分析、MD5 讨论、对比分析
4	中文大文件压缩	20	斗破苍穹 + 完美世界 压缩解压一致、大文件不崩
5	自适应流式压缩	20	一次运行处理 3 个文件、累积频率正确、生成 9 个文件
6	信息熵分析	5	熵和平均码长计算正确、对比分析合理
合计		100	

六、你需要提交什么

6.1 必须提交的材料

材料	说明
完整源代码	所有源文件，能直接编译运行
README.md	运行环境、怎么编译（附编译命令）、怎么用、每个命令的示例
实验报告（PDF）	内容见下方 6.2
测试输出文件	各测试文件生成的 <code>.huff</code> 、 <code>_freq.txt</code> 、 <code>_unzip.txt</code> 等（放 <code>output/</code> 目录）

6.2 实验报告要写什么

1. **算法设计** — 怎么统计频率、怎么建 Huffman 树（堆怎么用）、怎么生成编码表、怎么压缩和解压。
2. **两种哈希函数** — 选了哪两种、怎么实现的、碰撞概率分析。
3. **测试结果** —
 - 每个文件压缩前/后的大小、压缩率、耗时。
 - 解压后是否和原文件一致（附哈希值验证结果）。
 - 自适应模式下三个阶段各自的压缩率对比。
4. **信息熵分析** — 每个文件的信息熵、平均码长、编码效率。对比 `hamlet.txt` 和 `斗破苍穹.txt`，分析差异原因。
5. **复杂度分析** — 各操作的时间复杂度（ n = 文件字节数， m = 字符种类 ≤ 256 ）。
6. **总结** — 遇到了什么问题、怎么解决的、有什么收获。

七、提交前自查清单

- ☐ 1. 三个命令 `Huffman -c` / `Huffman -x` / `Huffman -a` 都能正常运行
- ☐ 2. 所有输出文件命名正确（`.huff`、`_freq.txt`、`_unzip.txt`、`_adp.huff`）

- ☐ 3. 输出文件和原文件在同一目录下
- ☐ 4. `hamlet.txt` 压缩解压后和原始文件完全一致 (`diff` 或哈希验证)
- ☐ 5. Huffman 树用了优先队列 / 小根堆构建, 不是每次 $O(n)$ 扫描
- ☐ 6. 两个哈希函数都是自己实现的, 不是直接调 `std::hash`
- ☐ 7. `斗破苍穹.txt` (16MB) 和 `完美世界.txt` (21MB) 都能成功压缩解压, 程序不崩
- ☐ 8. `Huffman -a <目录>` 一次运行生成全部 9 个文件
- ☐ 9. 自适应模式的阶段二、三确实用了累积频率来建树
- ☐ 10. `.huff` 文件包含频率表等必要信息, 能独立解压还原 (不依赖原始文件)
- ☐ 11. 报告里有信息熵、平均码长、编码效率计算和对比
- ☐ 12. 助教不看你的代码也能根据 README 编译、运行、复现所有结果

八、附录：助教批量测试说明

助教将按以下流程进行自动化测试。你的程序只要符合前面的接口规范就能通过。

8.1 测试目录结构

```
test_suite/
├── hamlet.txt
├── 斗破苍穹.txt
├── 完美世界.txt
└── adaptive_data/
    ├── 1.txt
    ├── 2.txt
    └── 3.txt
```

8.2 测试步骤

```
# =====
# 测试 1: 基础压缩
# =====
Huffman -c test_suite/hamlet.txt
# 期望生成:
#   test_suite/hamlet.huff
#   test_suite/hamlet_freq.txt
#   test_suite/hamlet_unzip.txt

# =====
# 测试 2: 单独解压
# =====
rm test_suite/hamlet_unzip.txt
Huffman -x test_suite/hamlet.huff
# 期望生成: test_suite/hamlet_unzip.txt

# =====
```

```
# 测试 3: 正确性验证
# =====
diff test_suite/hamlet.txt test_suite/hamlet_unzip.txt
# 应该没有任何差异

# =====
# 测试 4: 中文大文件
# =====
Huffman -c test_suite/斗破苍穹.txt
Huffman -c test_suite/完美世界.txt

# =====
# 测试 5: 自适应流式压缩
# =====
Huffman -a test_suite/adaptive_data/
# 期望在 adaptive_data/ 下生成 9 个文件:
#   1_adp.huff  1_freq.txt  1_unzip.txt
#   2_adp.huff  2_freq.txt  2_unzip.txt
#   3_adp.huff  3_freq.txt  3_unzip.txt

# =====
# 测试 6: 验证所有解压文件
# =====
diff test_suite/hamlet.txt          test_suite/hamlet_unzip.txt
diff test_suite/斗破苍穹.txt        test_suite/斗破苍穹_unzip.txt
diff test_suite/完美世界.txt        test_suite/完美世界_unzip.txt
diff test_suite/adaptive_data/1.txt  test_suite/adaptive_data/1_unzip.txt
diff test_suite/adaptive_data/2.txt  test_suite/adaptive_data/2_unzip.txt
diff test_suite/adaptive_data/3.txt  test_suite/adaptive_data/3_unzip.txt
# 全部应该没有差异
```

8.3 自动评分脚本检查项

检查项	怎么检查
文件命名是否正确	检查生成的文件名是否符合规范
文件位置是否正确	检查输出是否在原文件同目录
压缩解压是否一致	diff 原始文件和_unzip.txt
自适应模式	是否生成 9 个文件 + 阶段二三频率表是否体现累积